

REDES NEURONALES

CONJUNTO DE ALGORITMOS DE APRENDIZAJE
SUPERVISADO PARA REGRESIÓN Y
CLASIFICACIÓN BINARIA (O MULTICLASE)
DEEP LEARNING

ORÍGENES

BASÁNDOSE EN ESTUDIOS PROVENIENTES DE LA BIOLOGÍA, EN 1958 FRANK ROSENBLATT PRESENTA EL PERCEPTRÓN (SOBRE IDEAS ANTERIORES DE MC CULLOCH Y PITTS DE 1943)

FUE LA PRIMERA 'PROTO' RED NEURONAL. SE TRATA DE UN CLASIFICADOR BINARIO O DISCRIMINADOR LINEAL

<https://web.archive.org/web/20180722124753/https://data-speaks.luca-d3.com/2018/07/historia-de-la-ia-frank-rosenblatt-y-el.html>

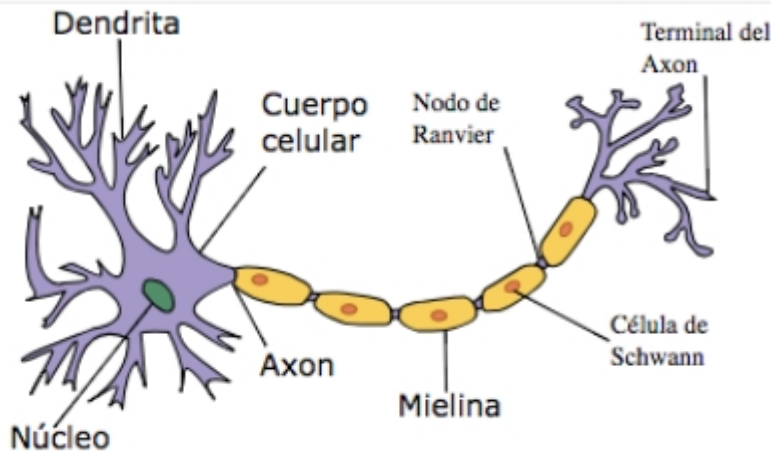
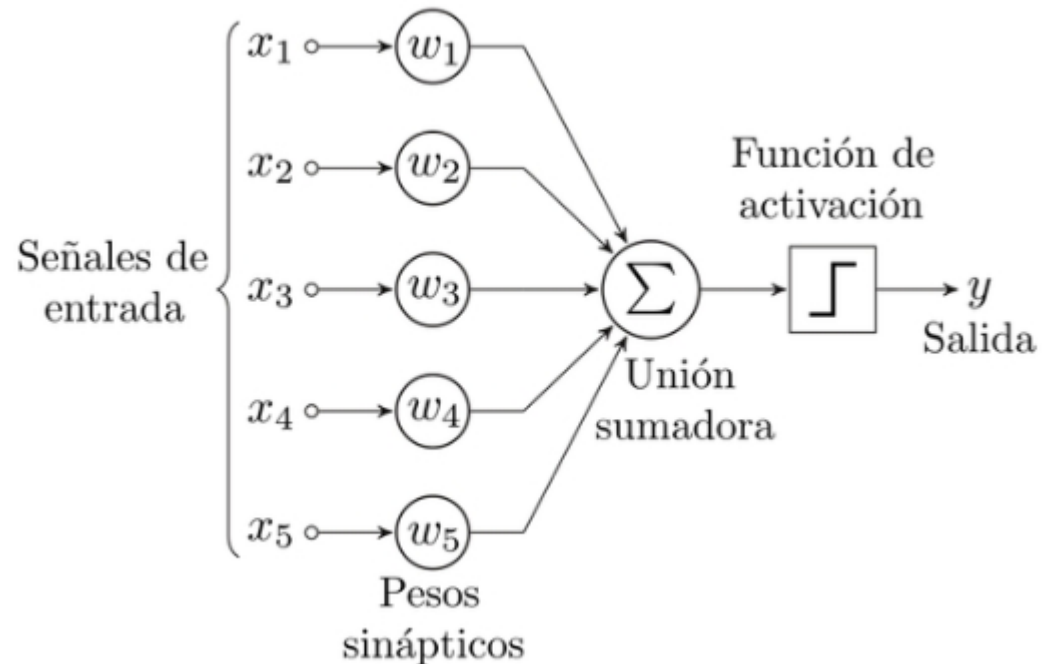
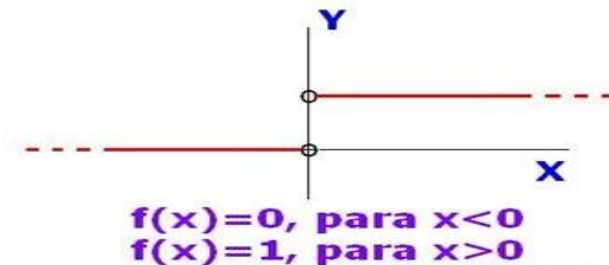


Figura 2. Detalle de una neurona del cerebro humano. Fuente: US Federal y



CÓMO FUNCIONA EL PERCEPTRÓN?

- LA INFORMACIÓN INGRESA A LA NEURONA Y CADA CARACTERÍSTICA x_i ES MULTIPLICADA POR UN PESO w_i (QUE PUEDE PENSARSE COMO UNA PERILLA QUE 'HABILITA' CADA VALOR CON UNA DADA 'INTENSIDAD')
- EN LA NEURONA SE SUMAN TODAS LAS CARACTERÍSTICAS 'PESADAS' POR LOS w_i -> $y = \text{SUMA}(w_i * x_i)$
- A LA SALIDA TENEMOS 'CASI' UNA REGRESIÓN LINEAL, PERO NOS FALTA APLICAR EL UMBRAL DE LA RESPUESTA DE SALIDA
- SI $\text{SUMA}(w_i * x_i) \leq \text{UMBRAL} \rightarrow y = 0$
- SI $\text{SUMA}(w_i * x_i) > \text{UMBRAL} \rightarrow y = 1$
- YA TENEMOS EL CLASIFICADOR!
- ESTE UMBRAL ES UNA FN 'ESCALÓN' (LLAMADA FUNCIÓN DE ACTIVACIÓN' POR LOS AMIGOS ;)

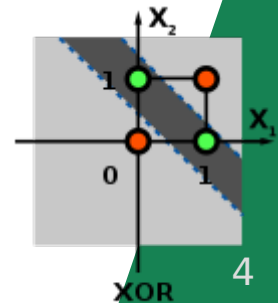
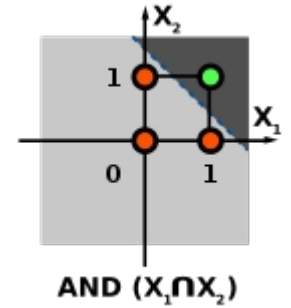
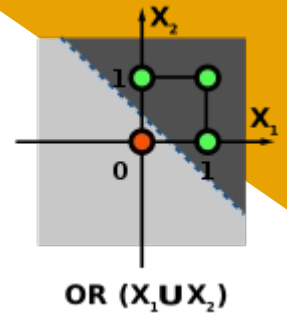


Función (Unitaria) de Heaviside

<http://dicio-mates.blogspot.com>

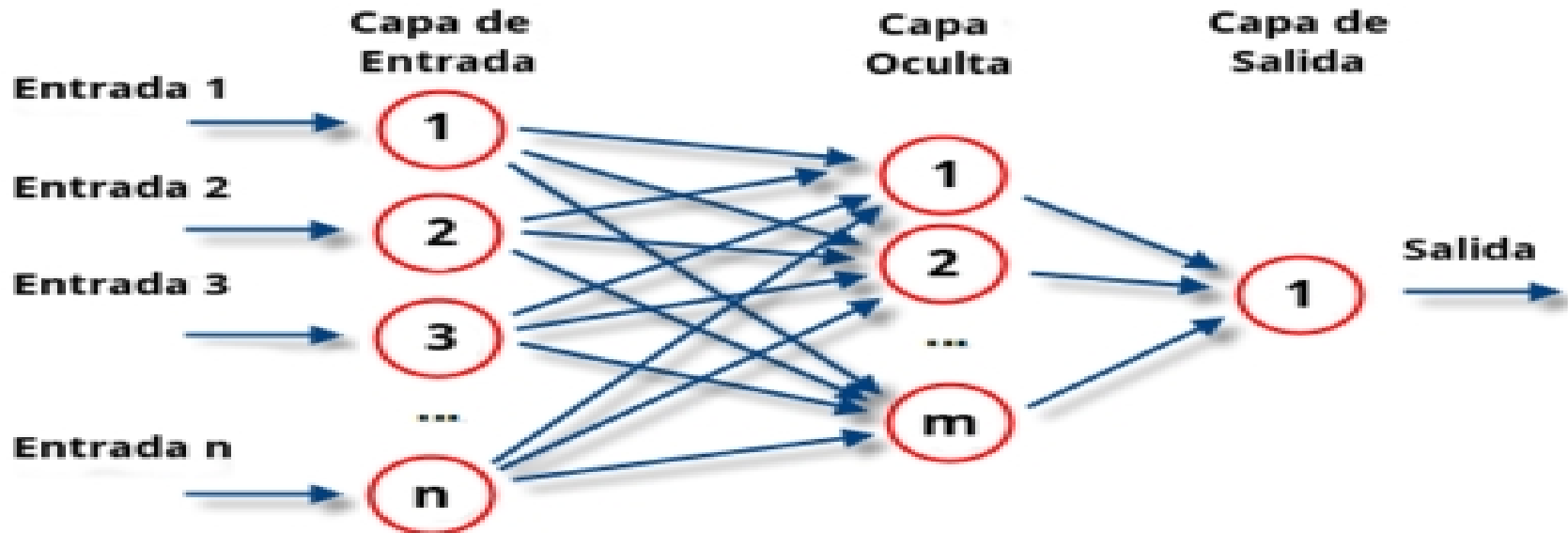
MÁS ALLÁ DEL PERCEPTRÓN

- EL PERCEPTRÓN NO ES CAPAZ DE RESOLVER PROBLEMAS QUE NO SON LINEALMENTE SEPARABLES... OTRA VEZ SOPA
- PERO... PODEMOS ENCADENAR VARIOS PERCEPTRONES (COMO LAS NEURONAS) Y CAMBIARLE LA FUNCIÓN DE ACTIVACIÓN Y AHÍ LA COSA CAMBIA!
- TENEMOS AL 'PERCEPTRÓN MULTICAPA' (MLP) QUE SÍ PUEDE CON ESTO ---->



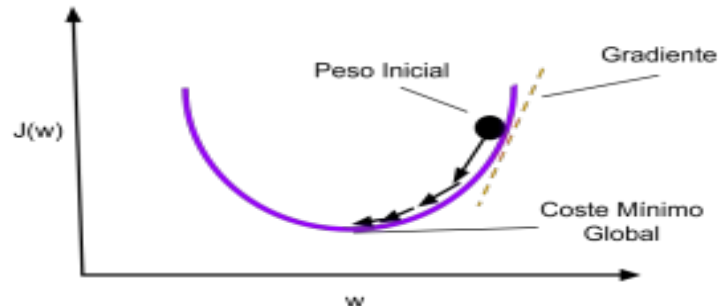
CÓMO CONECTAMOS A LOS PERCEPTRONES MULTICAPA?

- EN LA MISMA COLUMNA: OBTENEMOS UNA 'CAPA'
- EN LA MISMA FILA: CONECTAMOS CAPA CON CAPA



MIRANDO ADENTRO DE LA CAJA NEGRA...

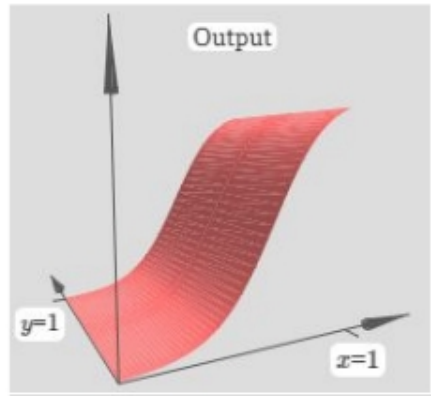
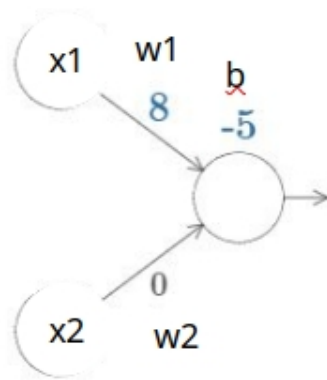
- TODO BIEN, PERO CÓMO LO HACE?
- PROPAGACIÓN HACIA ADELANTE: LOS PESOS w_i Y EL BIAS
- PROPAGACIÓN HACIA ATRÁS: PARA PODER APRENDER DE LOS DATOS, DEFINIMOS UNA FUNCIÓN DE COSTO (EL ERROR CUADRÁTICO MEDIO) QUE ES LA QUE DEBE HACERSE MÍNIMA (PARA QUE EL RESULTADO PREDICHO NO SE ALEJE DEL VERDADERO) ---> APARECE EL 'DESCENSO DEL GRADIENTE' COMO TÉCNICA PARA BUSCAR ESE MÍNIMO.



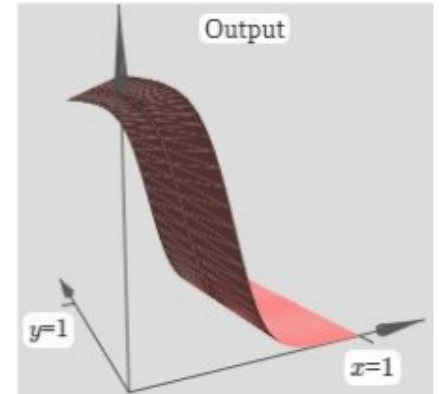
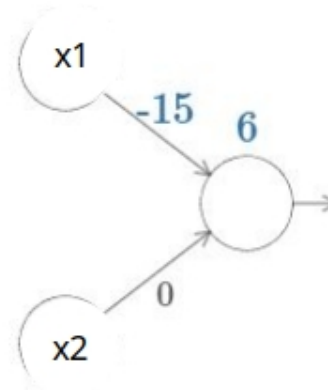
Los "w" y "b" afectan a la función de activación

La función de activación se aplica a la suma ponderada:

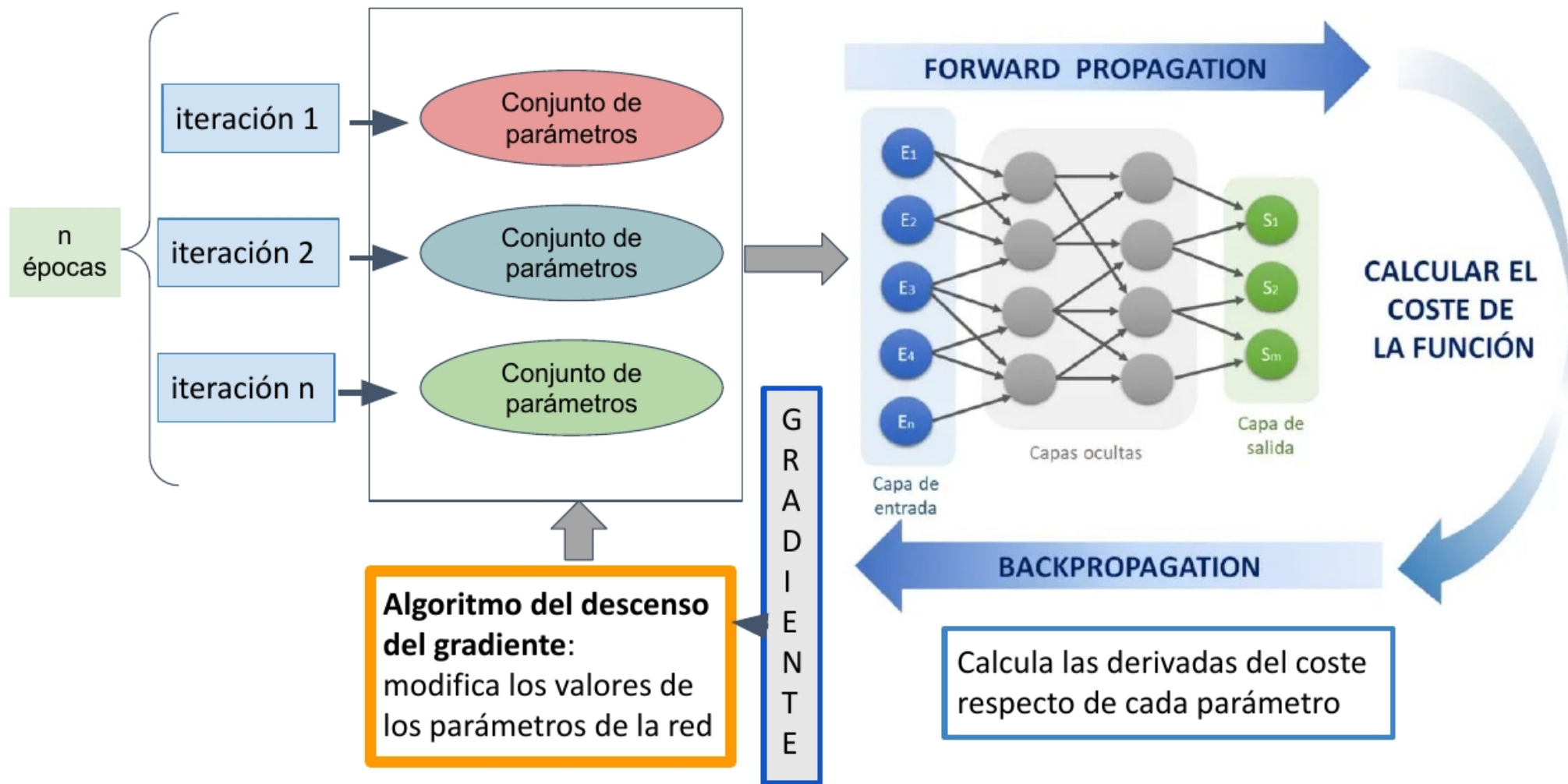
$$f(\sum_j w_j x_j + b)$$



.

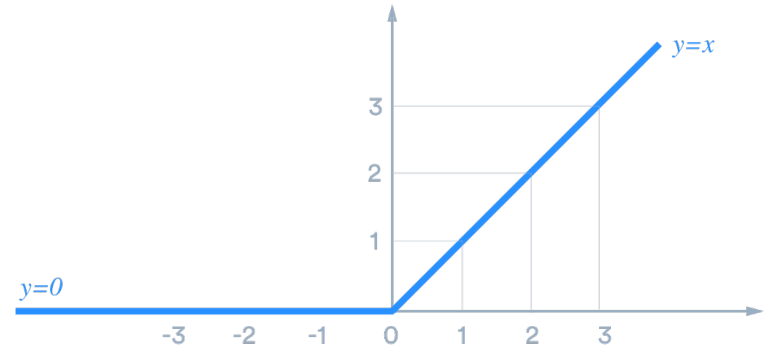
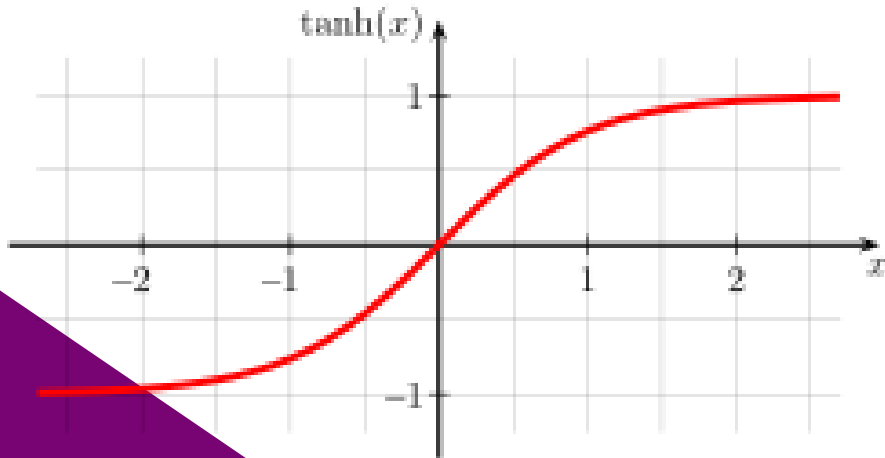
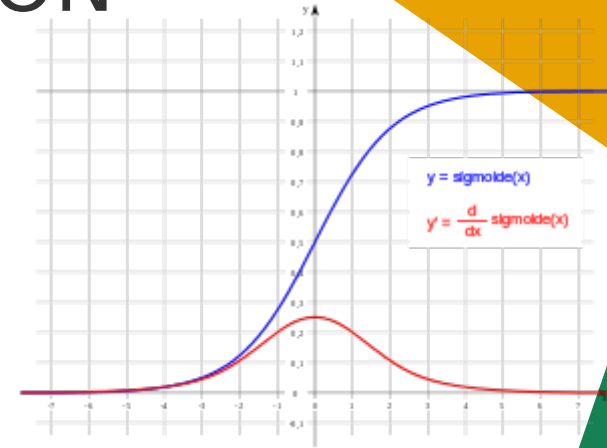


- Si modificamos "b" se desplaza la curva en el eje
- Si modificamos "w" se modifica la pendiente.

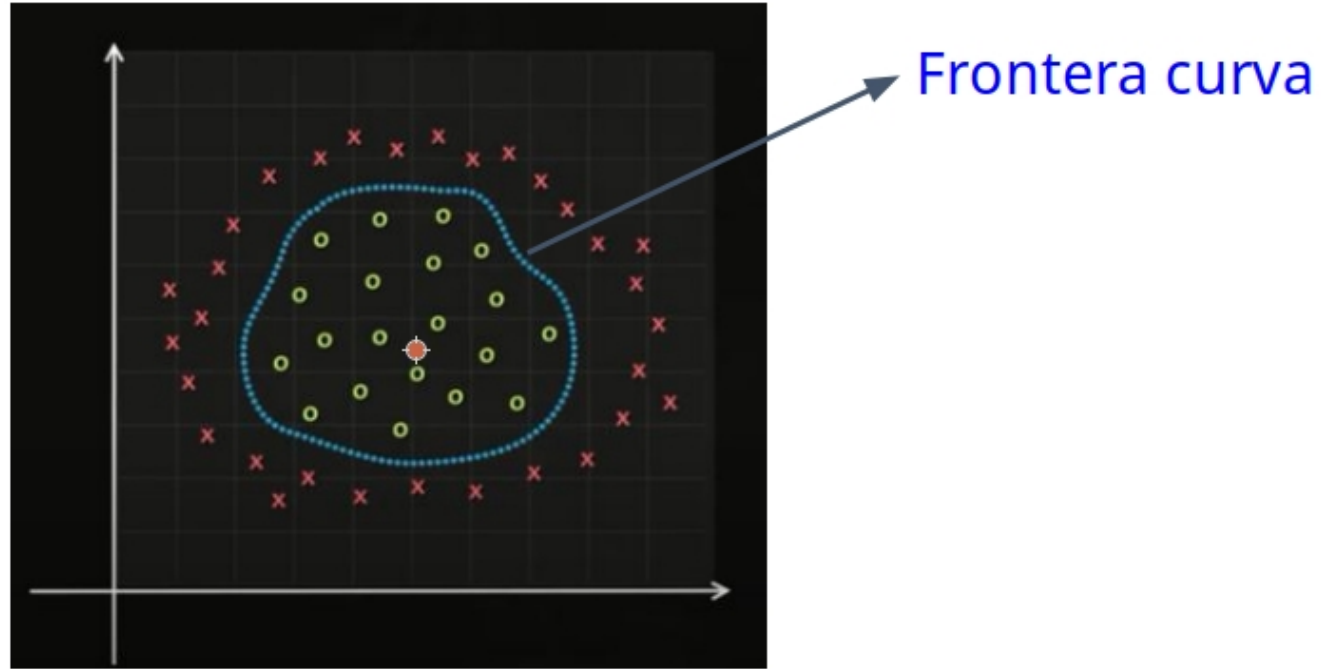


FUNCIONES DE ACTIVACIÓN

- SIGMOIDE: $f(t) = 1/(1 + \text{EXP}(-t))$
- TANH: $f(t) = (\text{EXP}(t) - \text{EXP}(-t)) / (\text{EXP}(t) + \text{EXP}(-t))$
- RELU: $f(t) = \max(0, t)$

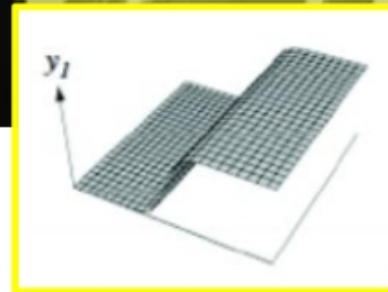
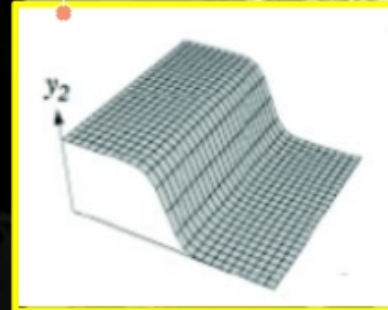
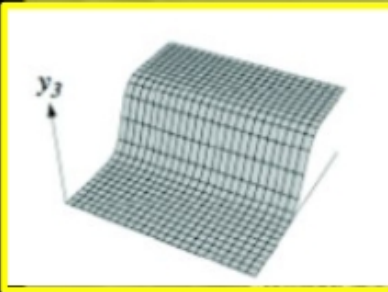
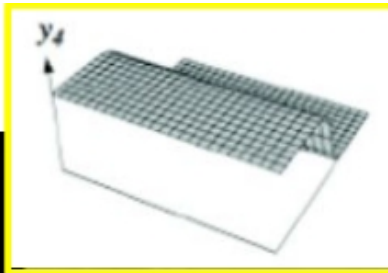


¿Cómo resolvemos esta situación?

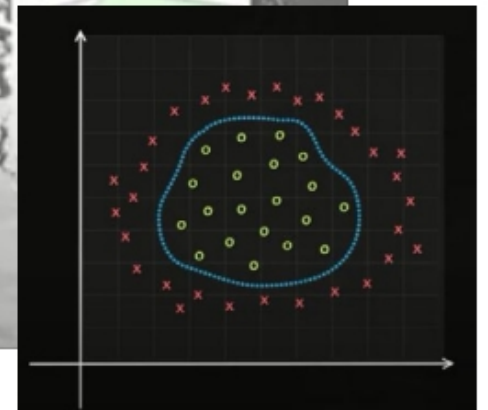
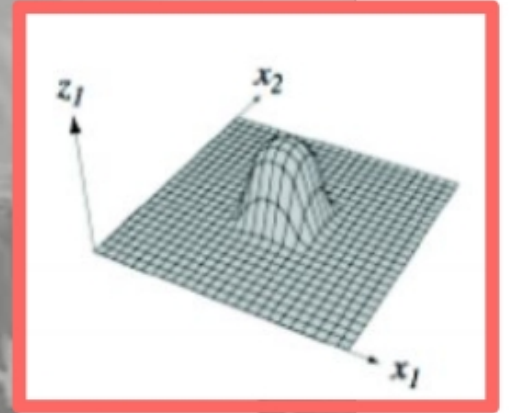


Lo podemos solucionar encadenando varias neuronas al mismo tiempo

Una solución posible



Combinación de las 4 figuras geomét



Problemas de las redes: Underfitting y Overfitting

Underfitting

Ocurre cuando **el modelo no aprende lo suficientemente** bien del problema. Tiene un rendimiento deficiente en el conjunto de datos de entrenamiento y no funciona bien en el de test

Overfitting

Ocurre cuando **el modelo aprende demasiado** el conjunto de datos de entrenamiento, con un buen rendimiento para el conjunto de datos de entrenamiento, pero no funciona bien con los datos de prueba.

Pasos para reducirlos

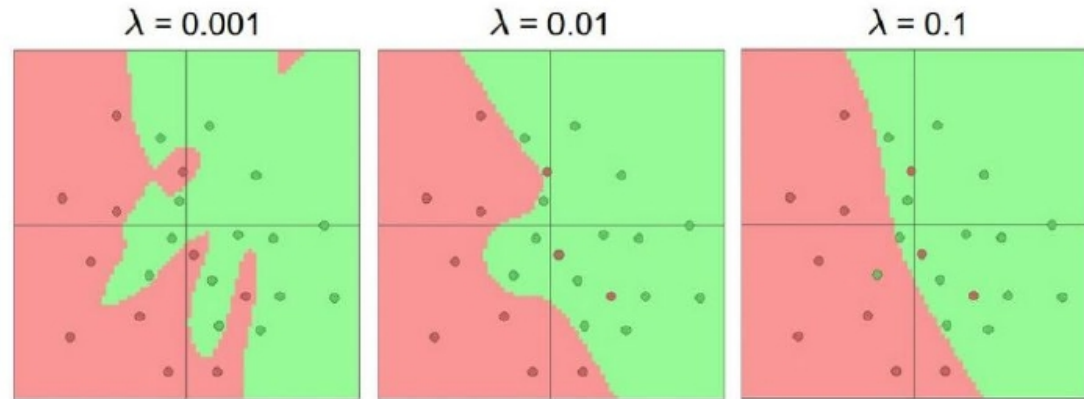
- Agregar más datos
- Usar data-augmentation (aumento de datos)
- Usar arquitecturas que generalicen bien
- Agregar regularización (Dropout, L1/L2, etc.)
- Reducir la complejidad de la arquitectura

La regularización

La regularización es un modo para restringir la complejidad de un modelo. Es útil cuando hay mucha correlación entre características, para filtrar ruido en los datos y también para evitar el Overfitting.

Aprendizaje= Loss + Penalización

$$Loss = \underbrace{\frac{1}{n} \sum_i^n E_i}_{\text{Error plano}} + \underbrace{\lambda \sum_j^m w_j^2}_{\text{Penalización}}$$

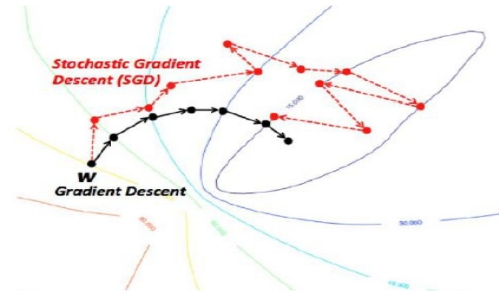


Regularización L2

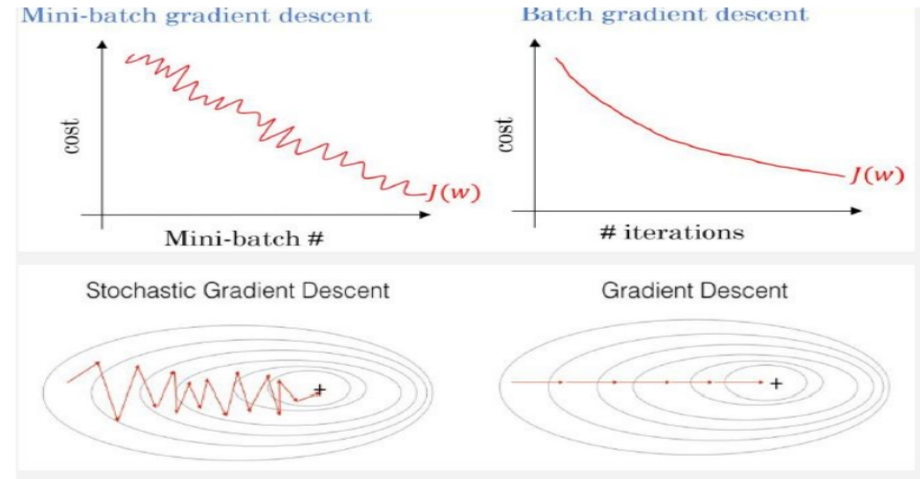
OPTIMIZADORES

- <https://velascoluis.medium.com/optimizadores-en-redes-neuronales-profundas-un-enfoque-pr%C3%A1ctico-819b39a3eb5>
- DESCENSO DEL GRADIENTE (TRADICIONAL)
- DESCENSO DEL GRADIENTE ESTOCÁSTICO (SGD)
- ADAGRAD
- RMSPROP
- ADAM

Descenso del gradiente: Tradicional vs Estocástico

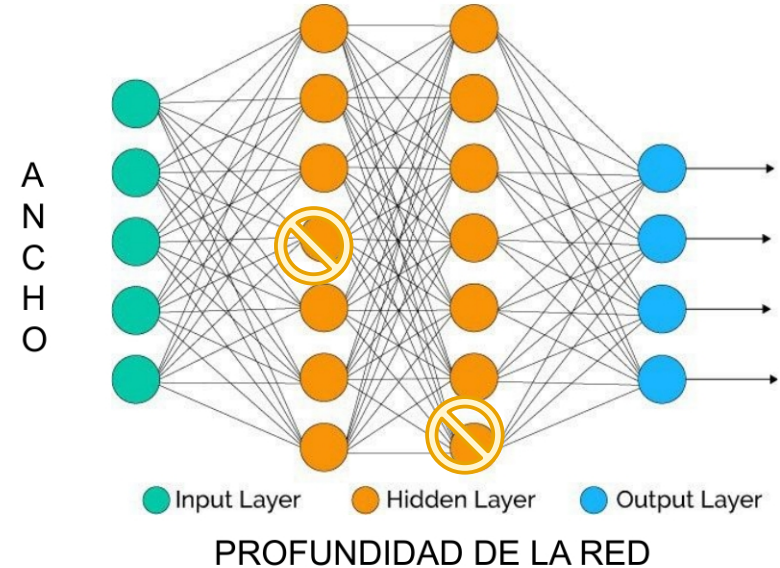
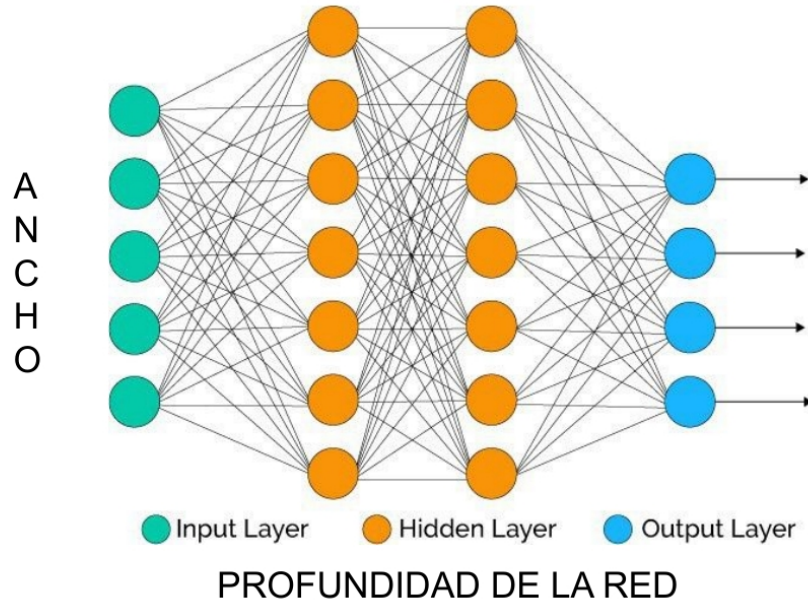


El descenso del gradiente clásico es lento y cae en mínimos locales.
El descenso del gradiente estocástico es más rápido.



DROPOUT

La primera red neuronal no es igual a la segunda. Aleatoriamente “apagamos” neuronas en cada época



Los hiperparámetros

Los hiperparámetros son los que guían el algoritmo de optimización de nuestro modelo.

No existe una configuración ideal, sino que depende del modelo y del dataset específico.

- Cantidad de capas
- Cantidad de neuronas por capa.
- Función de activación
- Tasa de aprendizaje (Learning rate (alfa))
- Batch size
- Épocas
- Regularización
- Dropout
- Early Stopping
- Callbacks
- Optimizador

BREVE HISTORIA

- 1958: Perceptrón (sobre ideas de la década del 40)
- 1980's: Funciones de Activación Sigmoidales
- Redes Feedforward
- Backpropagation
- 1989: Convolutional neural networks (CNN) / Recurrent neural networks (RNN)
- 1997: Long short term memory (LSTM)
- 2006: Deep Belief Networks (DBN): Nace deep learning
- Restricted Boltzmann Machine
- Encoder / Decoder = Auto-encoder
- 2014: Generative Adversarial Networks (GAN)
- 2017: Transformers
- 2022: ChatGPT
- 2025: vamos por la versión 5 de ChatGPT que se puede descargar como app en los celulares y hay IA hasta en la sopa...
- el futuro ???????

USOS

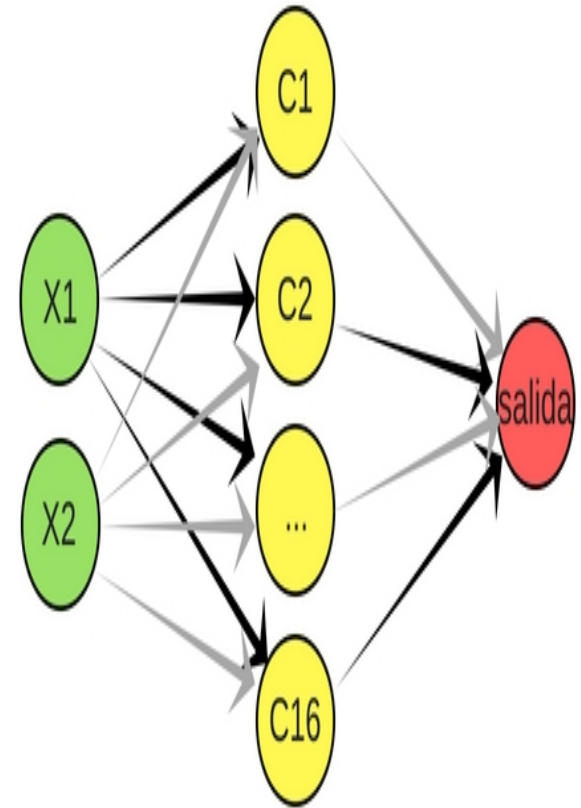
- UN MONTÓN!
- CLASIFICACIÓN BINARIA Y MULTICLASE
- REGRESIÓN
- CLASIFICACIÓN DE IMÁGENES
- DETECCIÓN DE OBJETOS
- PROCESAMIENTO LENGUAJE NATURAL
- PRONÓSTICO SERIES TEMPORALES
- Y UN LARGO ETCÉTERA...

LIBRERÍAS EN PYTHON

- TENEMOS VARIAS:
- SKLEARN: https://scikit-learn.org/stable/modules/neural_networks_supervised.html
- KERAS (TENSORFLOW): <https://keras.io/>
- PYTORCH: <https://pytorch.org/>

EJEMPLOS

- EJEMPLO XOR CON KERAS:
<https://www.aprendemachinelearning.com/una-sencilla-red-neuronal-en-python-con-keras-y-tensorflow/>
- EJEMPLO XOR CON NUMPY:
<https://towardsdatascience.com/implementing-the-xor-gate-using-backpropagation-in-neural-networks-c1f255b4f20d>



- **PREGUNTAS????**
- **GRACIAS POR TU PARTICIPACIÓN!!!**